



CSI247: Data Structures

Lecture 09 – Searching arrays

Department of Computer Science

B. Gopolang (247-275)



Previous Lesson

- Algorithm growth rate
- Big-O notation

Today ...

Learning Objectives

- By the end of this lecture, students must be able to:
 - Apply and implement linear search algorithm
 - Apply and implement binary search algorithm

A scenario

- Imagine the 2025/26 CSI247 class list from ASAS
- We can traverse this list to perform some operations
- Examples:
 - Check whether student with ID number 202401234 is in the class list
 - Change name of student with ID 202301723 to “Thomas Jones”
 - Display details of student with ID 202409342
- The above operations require searching the class list

1. Searching

- Searching is one of the common task in programming
- It involves traversing a collection
 - The process of looking for a specific element in a collection
 - E.g.: checking if a certain student is in the class list
- The traversed collection may be:
 - Ordered (sorted), or
 - unordered

- Why search?
 - To determine if an element is in a collection
 - Or to display/modify some information about an element that is in a collection (or delete item, etc)
- There are several search algorithms
 - Of interest in this course:
 - Linear search
 - Binary search

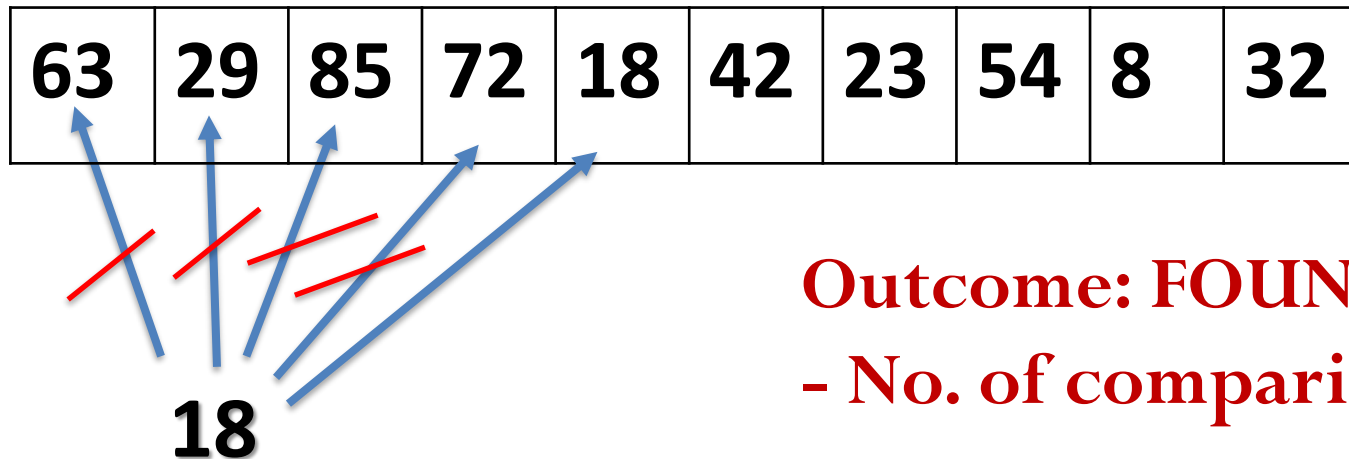
a) Linear Search

- A simple algorithm
- Aka sequential search
- Compares search key with each element in the array, sequentially
 - Until search key is found
 - Or we exhaust the collection

- What happens when search key is found?
 - Algorithm returns index of the element where the search item is stored
- If no match:
 - Algorithm returns -1.
- Note: For an ordered collection, there is no need to look at all elements
 - Can stop searching once we determine that the element cannot be in the collection

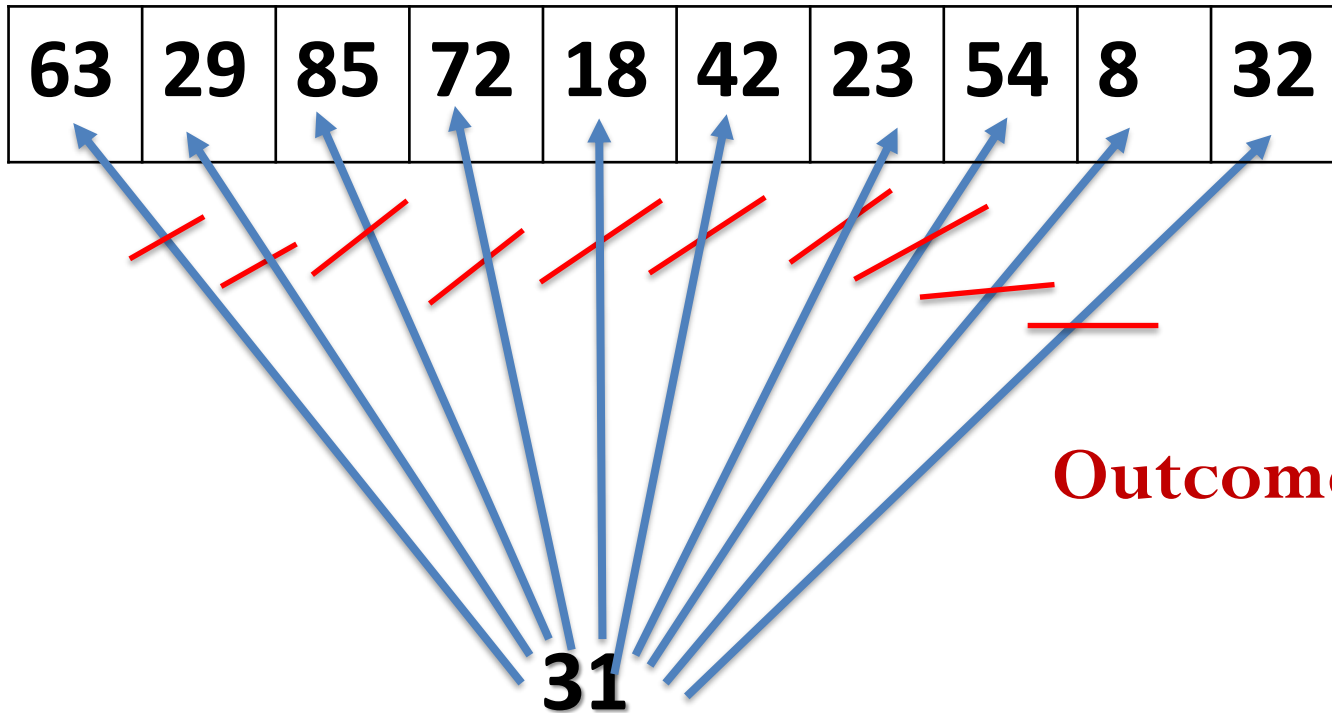
i) Searching an **unordered** collection

- Searching for 18:



Outcome: FOUND!
- No. of comparisons?

- Searching for 31?



Outcome: NOT FOUND

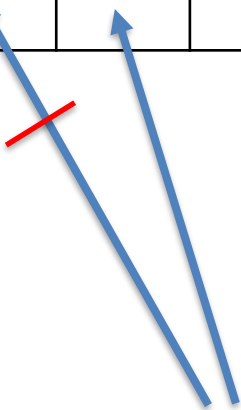


- No. of comparisons?
11

ii) Searching an **ordered** collection

- Searching for 18:

8	18	23	29	34	42	54	63	72	85
---	----	----	----	----	----	----	----	----	----

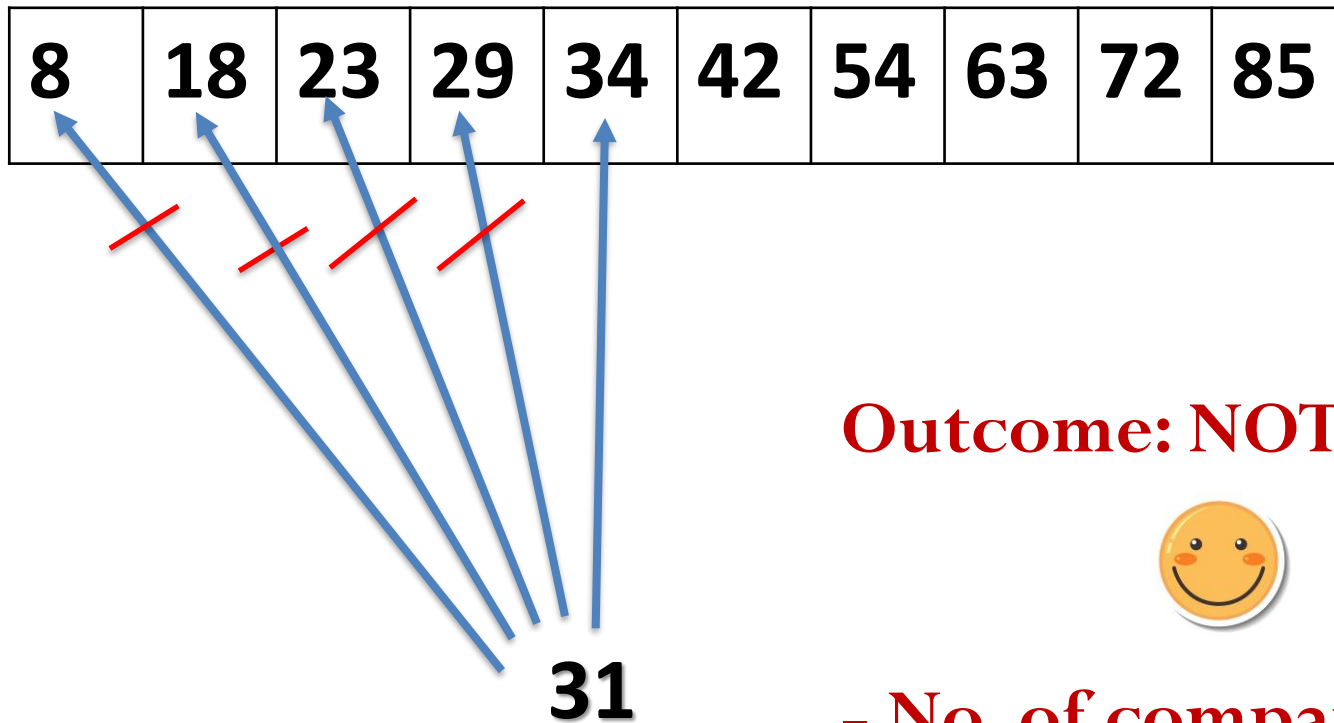


18

Outcome: FOUND!

- No. of comparisons?

- Searching for 31:



Outcome: NOT FOUND!



- No. of comparisons?

iii) Linear search implementation

- Write an **iterative** method, using linear search algorithm, that accepts an array of integers and an integer value, then return an integer representing the index of the specified value if it is found in the array. Otherwise, the method should return -1.
- Soln?
 - Next slide

Sample solution

```
public static int linearSearch (int arr[], int key) {  
    for (int i = 0; i < arr.length; i++) { // traverse all array items  
        if (arr[i] == key) // is value at arr[i] same as search key?  
            return i; // search key found  
    }  
    return -1; // search key not found  
}
```

iv) Linear search's time complexity

- Best-case runtime
 - When search item is at index 0
 - $O(1)$
- Worst-case runtime?
 - When item is not in the collection or it is stored at index number $N-1$
 - $O(n)$
- Convert the iterative method into a recursive one

Observation

- Sorted collection: searching is faster only if the search key is not in the list
 - Collection MUST be sorted
- In general, linear search's efficiency is similar when searching both unordered and ordered collections

Limitations

- Application areas
 - For dealing with small collections
 - For handling unsorted data
- Advantages
 - Easy to implement
- Limitations
 - It is not a fast algorithm

● Q:

- Isn't there a more efficient approach for ordered collections?

● A: There is!

- We need an approach that will reduce the number of comparisons!
- Hence more efficient!

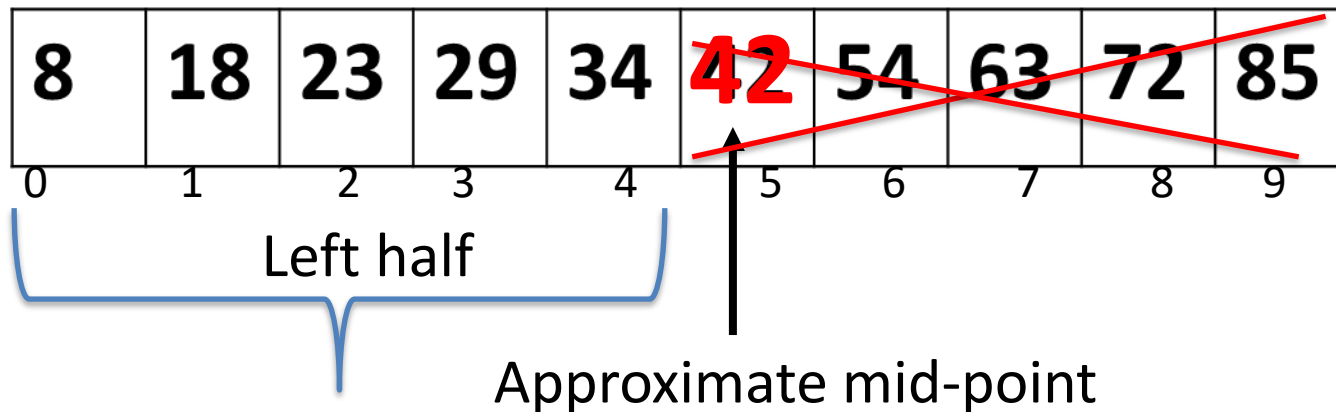
b) Binary Search

- Algorithm uses the “Divide and conquer” concept
 - It keeps on reducing search space by half
 - Until we find search key
 - OR we exhaust the collection (not found)
- Requires a sorted collection
- Because it uses an ordered (sorted) collection, we stop when we do not find the item where we expect to find it
- Good for data structures that allow for random access (E.g. arrays)

How does Binary Search operate?

- First, compare search key with the middle element
- K3 possible cases:
 - Search key == middle element
 - Found it, search ends
 - Search key < middle element
 - Search the key in the first half (Left) of the array.
 - Search key > middle element
 - Search in the second half (Right) of the array.

- Search for 18:

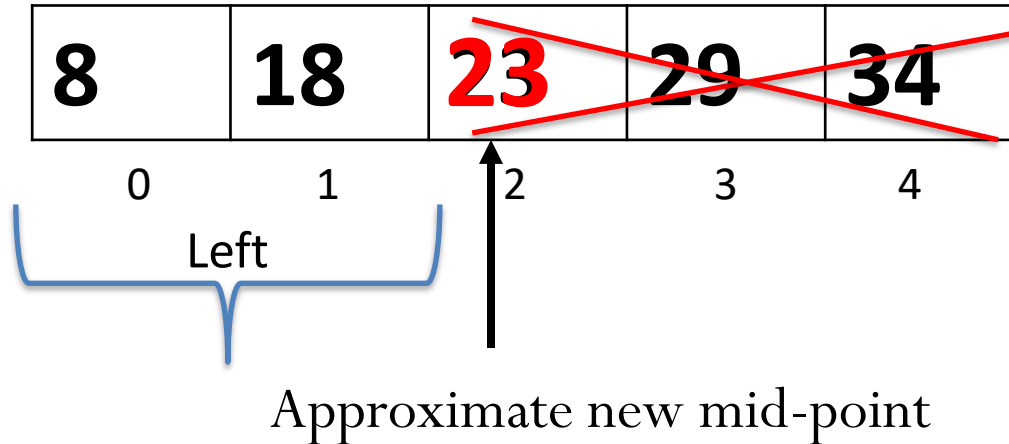


$18 == \text{mid-point key? NO.}$

$18 < \text{mid-point key? Yes.}$

focus the search on the L (Ignore R)

--- See next slide ---

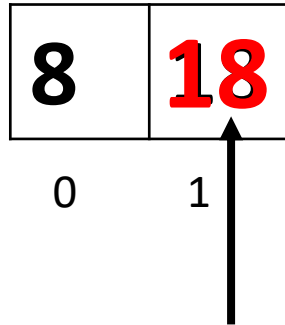


$18 == \text{mid-point key? NO.}$

$18 < \text{mid-point key? Yes.}$

focus the search on the L. (Ignore R)

--- See next slide ---



Approximate new midpoint

18 == midpoint key? YES.

Key found

Stop searching!

Observation

- Binary search reduces the work
 - It keeps on dividing the search space by 2
- We need to keep track of which “part” of the array to focus the search on

Binary search implementation

- Write an **iterative** method that accepts an array of integers, a search key and the relevant items, then return an integer representing the index where the value is stored if found. Otherwise, the method should return -1.
 - Soln? (see next slide)
- What is the algorithm's:
 - Best-case runtime? And when?
 - Worst-case runtime? And when?
- Convert the iterative method into a recursive one

Sample solution

```
public static int binarySearch (int[] arr, int key, int low, int high) {  
    int index = -1;           // assumption: search key not yet found  
    while (low <= high) {  
        int mid = low + (high - low) / 2; // calculate mid point  
        if (arr[mid] == key) { // is search key identical to item at mid point?  
            index = mid; // found the item  
            break;      // stop searching and return mid point  
        }  
        else if (arr[mid] < key)  
            low = mid + 1; // low = index after mid index (focus must shift to R half)  
        else  
            high = mid - 1; // high = index before mid index (focus must shift to L half)  
    }  
    return index; // search key not found  
}
```

Summary

- In this lesson, we covered 2 searching algorithms:
 - Linear search
 - Works with both unsorted and sorted arrays
 - Binary search
 - Array **MUST** be sorted

Exercises

- Assume array with contents (4, 12, 37, 50, 63, 81, 89, 101).
 - Using a) linear search and b) binary search algorithms,
 - Determine the number of comparisons required to search for:
 - 7
 - 37
 - 100
 - Note: You must demonstrate how you got each answer

Next lesson

- Sorting Arrays

Q & A

Thank you

